```
puts "Area of square = #{sq.area}"
```

produces:

```
A triangle has 3 sides
Area of square = 9
```

Both classes define a constant called SIDES and an instance method area, but these names don't get confused. You access the instance method via objects created from the class, and you access the constant by prefixing it with the name of the class followed by a double colon. The double colon (::) is Ruby's namespace resolution operator. The thing to the left must be a class or module, and the thing to the right is a constant defined in that class or module.

So, putting code inside a module or class is a good way of separating it from other code. Ruby's Math module is a good example—it defines constants such as Math::PI and Math::E and methods such as Math.sin and Math.cos. You can access these constants and methods via the Math module object:

```
Math::E                # => 2.718281828459045
Math.sin(Math::PI/6.0) # => 0.49999999999999994
```

(Modules have another significant use—they implement Ruby's *mixin* functionality, which we discussed in Mixins, on page 107.)

Ruby has an interesting little secret. The names of classes and modules are themselves just constants. Remember that we said that most everything in Ruby is an object. Well, classes and modules are, too. The name that you use for a class, such as String, is really just a Ruby constant containing the object representing that class. And that means that if you define classes or modules inside other classes and modules, the names of those inner classes are just constants that follow the same namespacing rules as other constants:

```
module Formatters
  class Html
    MEDIA_TYPE = "text/html"
    # ...
  end

  class Pdf
    # ...
  end
end

html_writer = Formatters::Html.new
html_media_type = Formatters::Html::MEDIA_TYPE
```

You can nest classes and modules inside other classes and modules to any depth you want (although it's rare to see them more than three deep).

So, now we know that we can use classes and modules to partition the names used by our programs.

Organizing Your Source Code

We have two related problems to solve: how do we split our source code into separate files, and where in the file system do we put those files?